

팍리스 실무형 일경험 프로그램

SDV 아키텍처 기반 자율주행 통합 시스템 구현 실습



5일차

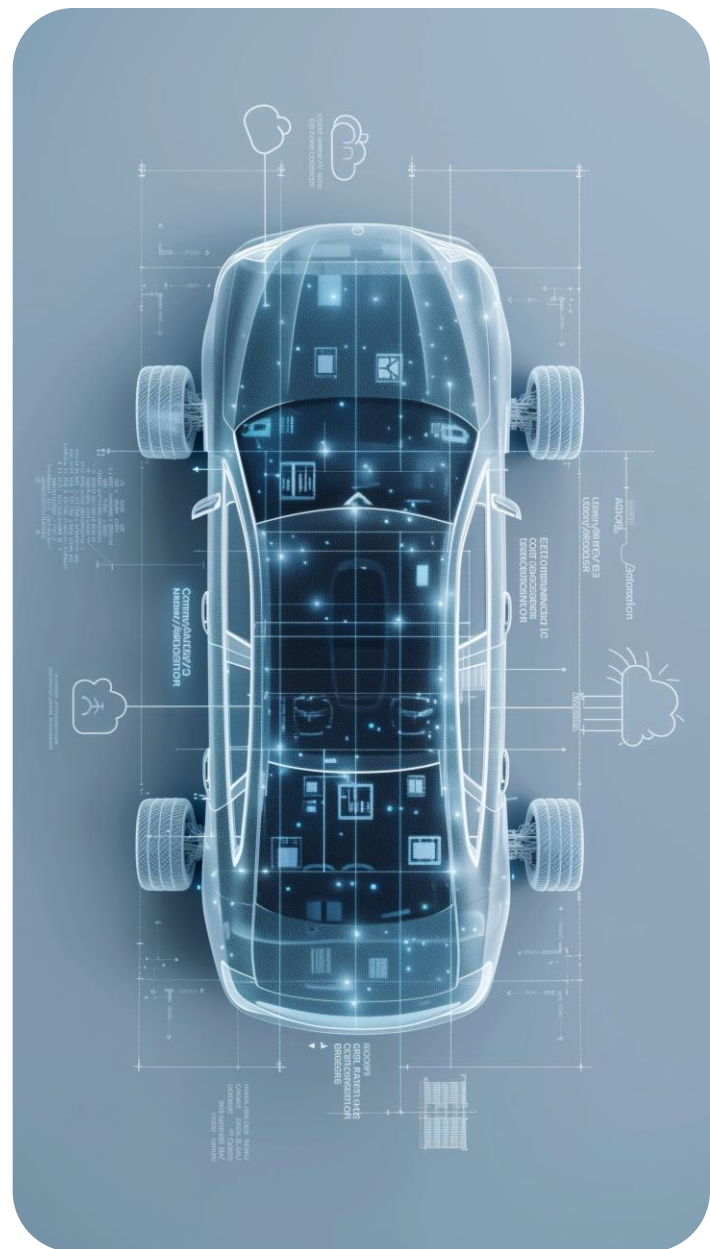
04 Control, HPC, Simulator Zone 통
합

Telechips TOPST

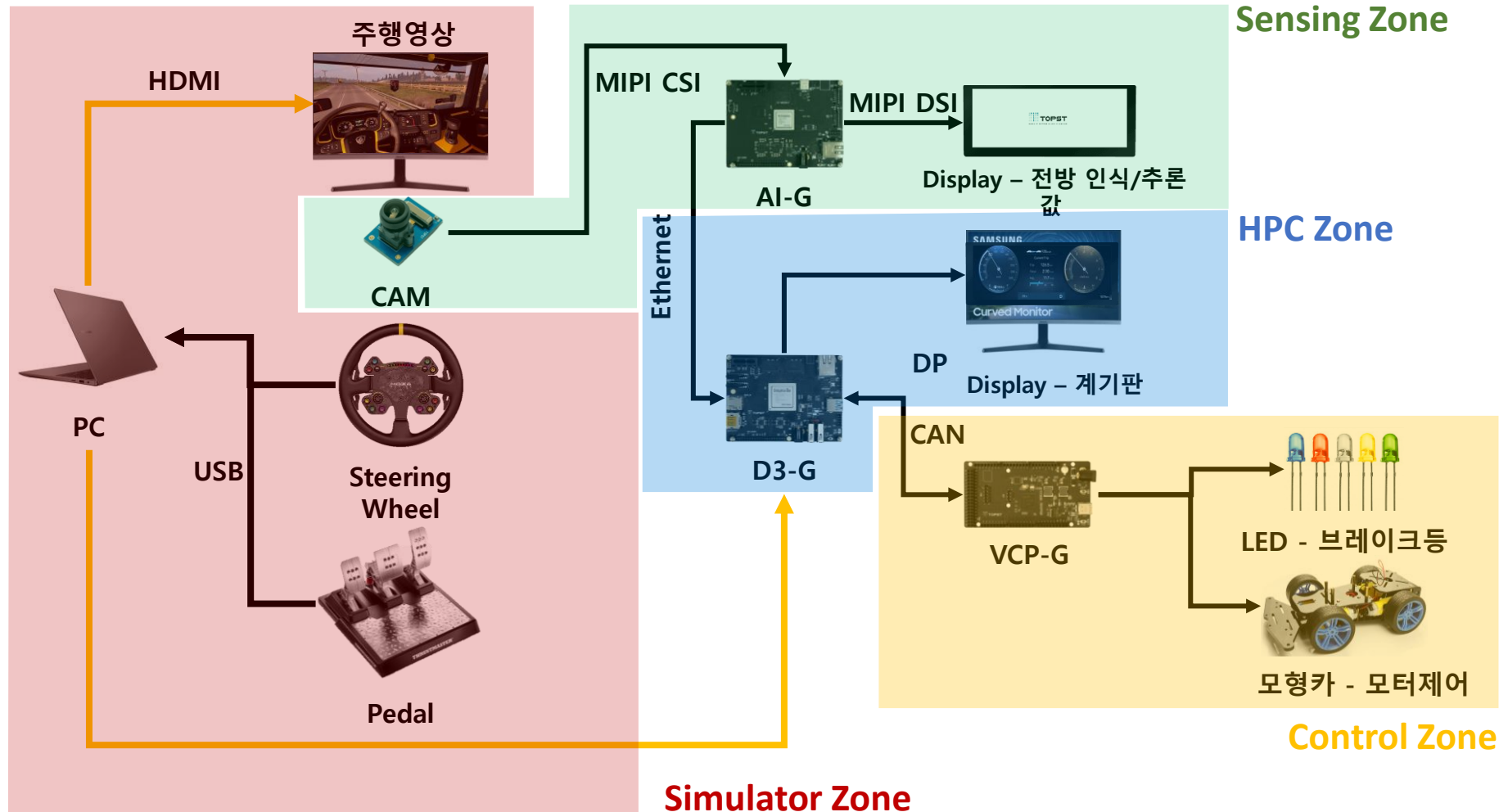


Table of Contents

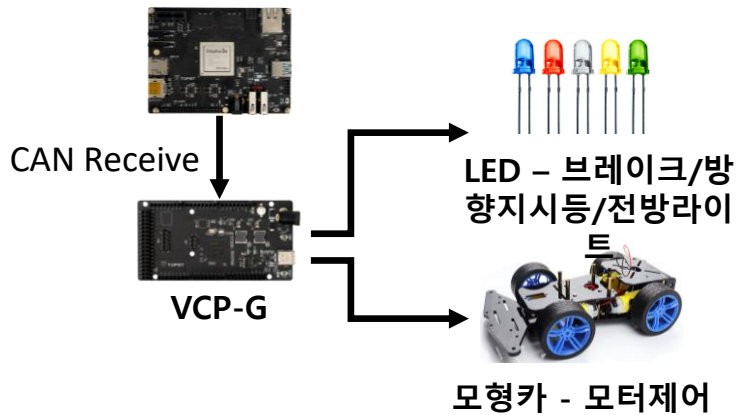
통합 Zone의 동작 구조 확인	01
Zone 별 연결 부분 완성	02
D3-G A72 실행 확인	03
D3-G R5 실행 확인	04
VCP-G 실행 확인	05
모형차와 Bread Board 동작 확인	06



Zonal 기반 차량 인터랙션 시스템-전체 시나리오



Control Zone 구현



Zone Description

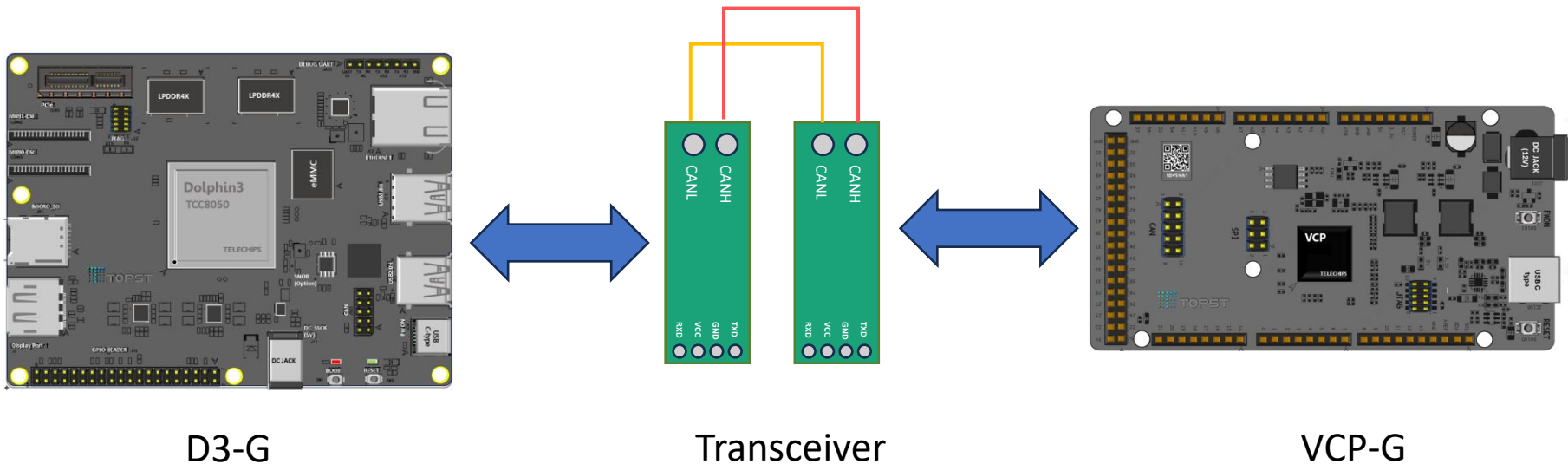
D3-G 로부터 수신한 CAN 메시지를 해석하여, 모형차 키트의 각 요소 제어(여러 LED, 모형카 모터제어 등)

Task Goal

1. CAN message receive
2. 모형차 Application 연동

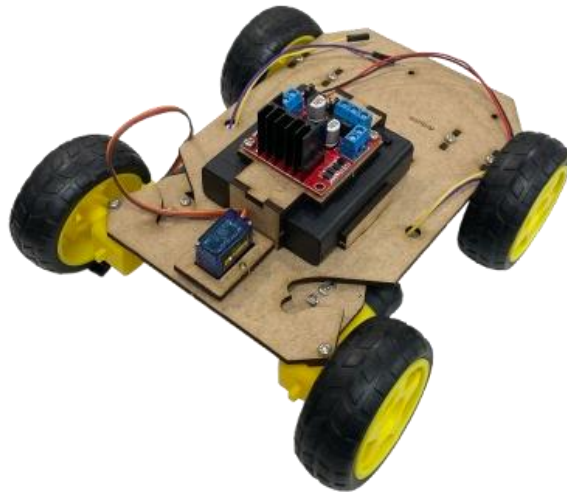
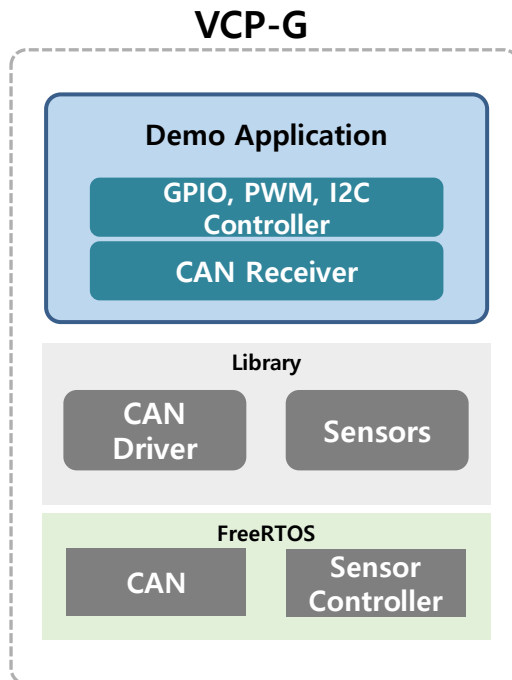
CAN Transceiver Connection

- D3-G와 VCP-G의 CAN 통신을 위한 CAN Transceiver 장비를 사용
- TJA1050 CAN 모듈
- 장치간 연결은 D304-HPCZone 챕터 참고



VCP에서 CAN 수신과 모형차 Kit 연동

- D3-G의 Transceiver 와 VCP-G CAN Port의 연결
- VCP-G와 Vehicle Kit 연결
 - 좌/우 방향 지시등
 - 전방 라이트
 - LCD 디스플레이
 - 모터

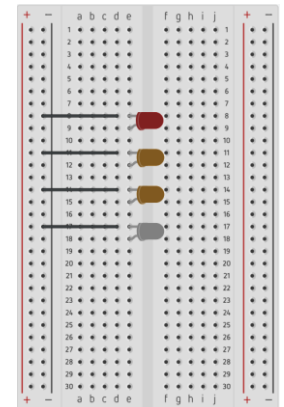


제공된 모형차 Kit

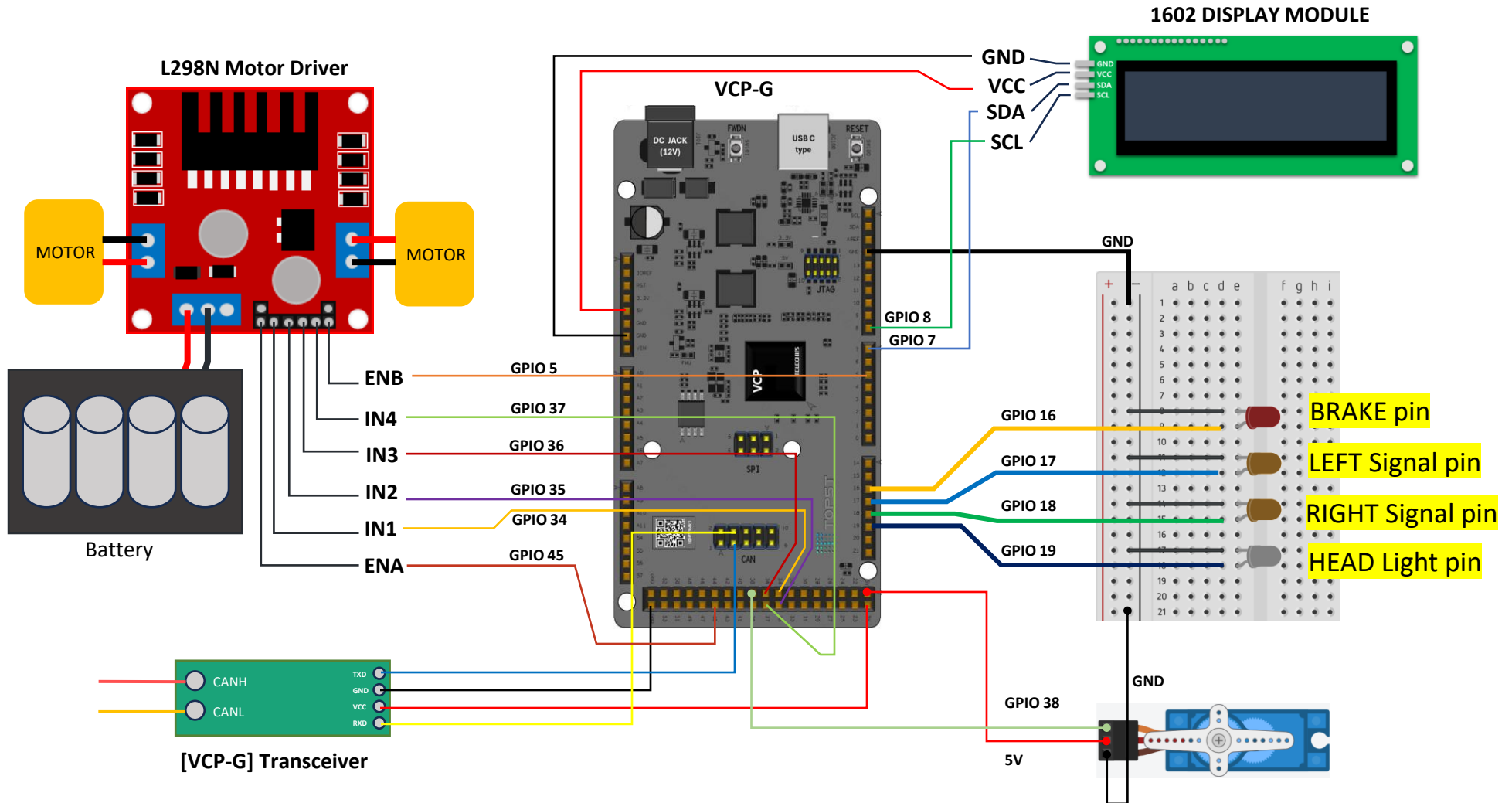
- L298N 모터드라이버*1EA
- Servo 모터*1EA
- DC 모터*4EA



1602A LCD Display * 1EA BreadBoard *1EA
LED * 4EA



VCP-G 와 모형차 Kit 회로 연결



VCP에서 CAN 수신 설명

- **can_demo.c 파일을 FreeRTOS 내에 함께 컴파일**
 - 오리엔테이션에서 받은 소스코드 참고 (sources/app.sample/app.can.demo 내에 소스코드 존재)
 - Telechips에서 제공하는 CAN 통신용 demo 프로그램
 - D3-G, VCP-G 에서 CAN에 접근하고 Data를 송/수신
 - VCP-G에서는 D3-G와 다르게 IPC 통신 내용은 없음
- **실행 옵션**
 - VCP-G 전원을 넣으면 사전에 업로드한 ROM 파일 실행
 - CAN data receive mode 자동 진입

VCP에서 CAN 수신 설명

```
CANErrorType_t CAN_Init
(
    void
)
{
    uint8          ucCh;
    CANController_t * psControllerInfo;
    CANErrorType_t  ret;

    psControllerInfo = NULL_PTR;
    ret              = CAN_ERROR_NONE;

    for( ucCh = 0 ; ucCh < CAN_CONTROLLER_NUMBER ; ucCh++ )
    {
        psControllerInfo = &CANDrvierInfo.dControllerInfo[ ucCh ];
        psControllerInfo->cChannelHandle = ucCh;

        ( void ) CAN_PortingDisableControllerInterrupts( psControllerInfo->cChannelHandle );

        ret = CAN_DrvInitChannel( psControllerInfo );

        if( ret != CAN_ERROR_NONE )
        {
            CAN_D( "[CAN]CAN_Init Failed \r\n", psControllerInfo->cChannelHandle );
            break;
        }

        ( void ) CAN_PortingEnableControllerInterrupts( psControllerInfo->cChannelHandle );
    }

    if( ret == CAN_ERROR_NONE )
    {
        CAN_D( "[CAN]CAN_Init Completed \r\n", psControllerInfo->cChannelHandle );
    }

    return ret;
} « end CAN_Init »
```

- CAN port 초기화
 - Driver Level 초기화
 - Register 직접 Setup
 - 3채널 GPIO 초기화
 - Controller Clock 초기화

VCP에서 CAN 수신 설명

```
static void CAN_DemoReceive
(
    uint8          ucCh
)
{
    uint32          uiRxMsgNum;
    CANMessage_t    sRxMsg;
    // CANErrorType_t ret;

    uiRxMsgNum      = 0;

    if( 0UL < gReceiveFlag[ ucCh ] )
    {
        while( 1 )
        {
            uiRxMsgNum = CAN_CheckNewRxMessage( ucCh );

            if( 0UL < uiRxMsgNum )
            {
                ( void ) CAN_GetNewRxMessage( ucCh, &sRxMsg );
            }
        }
    }
}
```

- CAN port에서 Data 수신

수신된 CAN Data 처리

- sRxMsg의 구조는 Send 측과 동일한 구조
- mId, mDataLength, mData 내용을 확인하고 Control Logic을 확정
- ControlBreadBoardSensors() 함수 call

```
mcu_printf( "[CAN DEMO]\n" );
mcu_printf( "< Channel %d Received Message Information >\n", ucCh );
mcu_printf( "*****\n" );
mcu_printf( "[ID] : 0x%X, [DATA SIZE] : %d, [DATA] : \r\n", sRxMsg.mId, sRxMsg.mDataLength );

for( ucMsgLength = 1U ; ucMsgLength < ( sRxMsg.mDataLength + 1U ) ; ucMsgLength++ )
{
    mcu_printf( "0x%02X ", sRxMsg.mData[ ucMsgLength - 1U ] );

    if( ( ucMsgLength % 16U ) == 0U ) {
        mcu_printf( "\n" );
    }
}

mcu_printf( "\n" );
mcu_printf( "*****\n" );
mcu_printf( "\n" );
ControlBreadBoardSensors( sRxMsg.mId, sRxMsg.mDataLength, sRxMsg.mData );
```

VCP에서 CAN 수신 설명

메시지 종류	ID	Data
브레이크등	0x101	[0] : on-0x01, off-0x02
방향지시등	0x102	[0] : on-0x01, off-0x02 [1] : left-0x01, right-0x02
비상지시등	0x103	[0] : on-0x01, off-0x02
전조등	0x104	[0] : on-0x01, off-0x02
연료레벨	0x105	[0] : 0%-0x00 ~ 100%-0x64
모터동작	0x106	[0] : 0%-0x00 ~ 100%-0x64

- ID와 Data값으로 별도 Logic 구현
 - 정의된 ID 및 Data 값에 의해 Sensor 제어 가능
- 수신 부의 동작 구현
 - Can_vcp_ctrl.c와 can_vcp_ctrl.h 에 구현

```
enum VCP_IO_TYPE {
    VCP_IO_BREAK_LIGHT=0x101,
    VCP_IO_TURN_SIGNAL=0x102,
    VCP_IO_EMER_SIGNAL=0x103,
    VCP_IO_HEAD_LIGHT=0x104,
    VCP_IO_FUEL_LEVEL=0x105,
    VCP_IO_MOTOR_SPEED=0x106
};
```

```
enum VCP_IO_ACTION {
    VCP_IO_ACTION_ON=0x01,
    VCP_IO_ACTION_OFF=0x02
};
```

```
enum VCP_IO_SUBTYPE {
    VCP_IO_SUB_LEFT=0x01,
    VCP_IO_SUB_RIGHT=0x02
};
```

VCP 센서 초기화

```
boolean InitSensorControls(void)
{
    MotorPWM_Init();
    I2C_Init();
    if (I2C_Open(I2C_CH, I2C_PORT, I2C_SPEED, NULL_PTR, NULL_PTR) != SAL_RET_SUCCESS)
    {
        mcu_printf("[LCD] I2C_Open failed (ch=%d, port=%d, speed=%d)\r\n",
                    (int)I2C_CH, (int)I2C_PORT, (int)I2C_SPEED);
        return FALSE;
    }

    lcd_init();

    return TRUE;
}
```

```
static PDMModeConfig_t cfgA, cfgB, cfgWheel;
void MotorPWM_Init(void)
{
    static boolean initied = FALSE;
    if (initied) return;

    /* Init Pin */
    pin_out(IN1); pin_out(IN2); pin_out(IN3); pin_out(IN4);
    pin_lo(IN1); pin_lo(IN2); pin_lo(IN3); pin_lo(IN4);

    /* Init PDM */
    PDM_Init();
    PDM_CfgSetWrPw();
    PDM_CfgSetWrLock(0);

    SAL_MemSet(&cfgA, 0, sizeof(cfgA));
    cfgA.mcPortNumber = ENA_PORT;
    cfgA.mcOperationMode = PDM_OUTPUT_MODE_PHASE_1;
    cfgA.mcInversedSignal = 0;
    cfgA.mcOutSignalInIdle = 0;
    cfgA.mcLoopCount = 0;
    cfgA.mcOutputCtrl = 0;
    cfgA.mcPeriodNanoSec1 = PWM_PERIOD_NS;
    cfgA.mcDutyNanoSec1 = 0;

    SAL_MemSet(&cfgB, 0, sizeof(cfgB));
    cfgB.mcPortNumber = ENB_PORT;
    cfgB.mcOperationMode = PDM_OUTPUT_MODE_PHASE_1;
    cfgB.mcInversedSignal = 0;
    cfgB.mcOutSignalInIdle = 0;
    cfgB.mcLoopCount = 0;
    cfgB.mcOutputCtrl = 0;
    cfgB.mcPeriodNanoSec1 = PWM_PERIOD_NS;
    cfgB.mcDutyNanoSec1 = 0;

    (void)pdm_apply(ENA_SEL, &cfgA);
    (void)pdm_apply(ENB_SEL, &cfgB);

    initied = TRUE;
} « end MotorPWM_Init »
```

VCP 센서 초기화

```

void lcd_send(uint8 mode, uint8 data)
{
    uint8 high = data & 0xF0;
    uint8 low  = (data << 4) & 0xF0;
    uint8 buf[6];

    // BL=0x08, EN=0x04, RS=0x01 (RW=0)
    buf[0] = high | 0x08 | (mode ? 0x01 : 0x00);
    buf[1] = high | 0x0C | (mode ? 0x01 : 0x00);
    buf[2] = high | 0x08 | (mode ? 0x01 : 0x00);
    buf[3] = low  | 0x08 | (mode ? 0x01 : 0x00);
    buf[4] = low  | 0x0C | (mode ? 0x01 : 0x00);
    buf[5] = low  | 0x08 | (mode ? 0x01 : 0x00);

    I2CXfer_t xfer = {
        .xCmdLen = 0,
        .xOutLen = 6,
        .xOutBuf = buf,
        .xInLen = 0,
        .xInBuf = NULL,
        .xCmdBuf = NULL,
        .xOpt = 0
    };

    (void)I2C_Xfer(I2C_CH, (uint8)(LCD_ADDR << 1), xfer, 0); // sync
    SAL_TaskSleep(2);
} « end lcd_send »

void lcd_cmd(uint8 cmd)
{
    lcd_send(LCD_CMD, cmd);
}

void lcd_init(void)
{
    SAL_TaskSleep(50);
    lcd_cmd(0x33);
    lcd_cmd(0x32);
    lcd_cmd(0x28);
    lcd_cmd(0x0C);
    lcd_cmd(0x06);
    lcd_cmd(0x01);
    SAL_TaskSleep(5);
}

```

브레이크등 제어

```
switch(mId)
{
    case VCP_IO_BREAK_LIGHT:
        ucMsgData[0] = pucData[0];
        if(ucMsgData[0] == VCP_IO_ACTION_ON)
        {
            // turn on the break light
            ControlBrakeLight(true);
        }
        else
        {
            // turn off the break light
            ControlBrakeLight(false);
        }

        break;
}
```

• 브레이크등 처리

- ID = 0x101
- On : [0] - 0x01
- Off : [0] - 0x02

```
#define BrakeLEDPIN GPIO_GPA(6)
static void BrakeLED_ON(void) { GPIO_Set(BrakeLEDPIN, 1); }
static void BrakeLED_OFF(void) { GPIO_Set(BrakeLEDPIN, 0); }
static void ControlBrakeLight(bool bTurnOn)
{
    boolean binit = FALSE;
    if (binit == FALSE) {
        GPIO_Config(BrakeLEDPIN, GPIO_FUNC(0) | GPIO_OUTPUT | GPIO_NOPULL | GPIO_DS(3) | GPIO_INPUTBUF_DIS);
        BrakeLED_OFF();
        binit = TRUE;
    }

    if (bTurnOn) {
        BrakeLED_ON();
        mcu_printf("[BRAKE] ON\r\n");
    } else {
        BrakeLED_OFF();
        mcu_printf("[BRAKE] OFF\r\n");
    }
}
```

- ControlBrakeLight 함수가 호출됨에 따라 수신된 값에 의해 GPIO LED 제어

방향지시등 제어

```
case VCP_IO_TURN_SIGNAL:
    ucMsgData[0] = pucData[0];
    ucMsgData[1] = pucData[1];

    if(ucMsgData[1] == VCP_IO_SUB_LEFT)
    {
        // turn off the right signal
        ControlSignalLight(false, false);
        //~

        if(ucMsgData[0] == VCP_IO_ACTION_ON)
        {
            // turn on the left signal
            ControlSignalLight(true, true);
            //~
        }
        else
        {
            // turn off the left light
            ControlSignalLight(true, false);
            //~
        }
    }
    else
    {
        // turn off the left signal
        ControlSignalLight(true, false);
        //~

        if(ucMsgData[0] == VCP_IO_ACTION_ON)
        {
            // turn on the right signal
            ControlSignalLight(false, true);
            //~
        }
        else
        {
            // turn off the right light
            ControlSignalLight(false, false);
            //~
        }
    }
} « end else »

break;
```

• 방향지시등 처리

- ID = 0x102
- Left : [1] – 0x01
- Right : [1] – 0x02
- On : [0] – 0x01
- Off : [0] – 0x02

방향지시등 제어

```
/* ----- Turn Signal Light ----- */
void LeftSignalled_ON(void)
{
    GPIO_Set(LeftSignalledPIN, 1);
}

void LeftSignalled_OFF(void)
{
    GPIO_Set(LeftSignalledPIN, 0);
}

void RightSignalled_ON(void)
{
    GPIO_Set(RightSignalledPIN, 1);
}

void RightSignalled_OFF(void)
{
    GPIO_Set(RightSignalledPIN, 0);
}

void ControlSignalLight(boolean bLeft, boolean bTurnOn)
{
    static boolean binit = FALSE;

    mcu_printf("SIGNAL] Controlling Signal Light\r\n");
    if(binit == FALSE) {
        GPIO_Config(LeftSignalledPIN, GPIO_FUNC(0) | GPIO_OUTPUT | GPIO_NOPULL | GPIO_DS(3) | GPIO_INPUTBUF_DIS);
        GPIO_Config(RightSignalledPIN, GPIO_FUNC(0) | GPIO_OUTPUT | GPIO_NOPULL | GPIO_DS(3) | GPIO_INPUTBUF_DIS);
        LeftSignalled_OFF();
        RightSignalled_OFF();
        binit = TRUE;
    }

    if (bLeft) {
        if (bTurnOn) {
            LeftSignalled_ON();
            mcu_printf("[LeftSignal] ON\r\n");
        }
        else {
            LeftSignalled_OFF();
            mcu_printf("[LeftSignal] OFF\r\n");
        }
    }
    else {
        if (bTurnOn) {
            RightSignalled_ON();
            mcu_printf("[RightSignal] ON\r\n");
        }
        else {
            RightSignalled_OFF();
            mcu_printf("[RightSignal] OFF\r\n");
        }
    }
}

« end ControlSignalLight »
```


비상지시등 제어

```
case VCP_IO_EMER_SIGNAL:
    ucMsgData[0] = pucData[0];
    if(ucMsgData[0] == VCP_IO_ACTION_ON)
    {
        // turn on the emergency signal lights
        ControlSignalLight(true, true);
        ControlSignalLight(false, true);
    }
    else
    {
        // turn off the emergency signal lights
        ControlSignalLight(true, false);
        ControlSignalLight(false, false);
    }

    break;
```

• 비상지시등 처리

- ID = 0x103
- On : [0] - 0x01
- Off : [0] - 0x02

전조등 제어

```
case VCP_IO_HEAD_LIGHT:
    ucMsgData[0] = pucData[0];
    if(ucMsgData[0] == VCP_IO_ACTION_ON)
    {
        // turn on the head light
        ControlHeadLight(true);
    }
    else
    {
        // turn off the head light
        ControlHeadLight(false);
    }

    break;
```

• 전조등 처리

- ID = 0x104
- On : [0] - 0x01
- Off : [0] - 0x02

전조등 제어

```
/* ----- Head Light ----- */

void HeadLED_ON(void)
{
    GPIO_Set(HeadLEDPIN, 1);
}

void HeadLED_OFF(void)
{
    GPIO_Set(HeadLEDPIN, 0);
}

void ControlHeadLight(boolean bTurnOn)
{
    static boolean binit = FALSE;

    mcu_printf("[LIGHT] Controlling Head Light\r\n");
    if (binit == FALSE) {
        GPIO_Config(HeadLEDPIN, GPIO_FUNC(0) | GPIO_OUTPUT | GPIO_NOPULL | GPIO_DS(3) | GPIO_INPUTBUF_DIS);
        HeadLED_OFF();
        binit = TRUE;
    }

    if (bTurnOn) {
        HeadLED_ON();
        mcu_printf("[Head Light] ON\r\n");
    } else {
        HeadLED_OFF();
        mcu_printf("[Head Light] OFF\r\n");
    }
}
```

연료레벨 표시 제어

```
case VCP_IO_FUEL_LEVEL:  
    ucMsgData[0] = (pucData[0] > 100)?100:pucData[0];  
    // set the fuel level  
    ControlFuelLevel(ucMsgData[0]);  
    //~  
    break;
```

- 연료 레벨 처리

- ID = 0x105
- 0% : [0] - 0x00
- 100% : [0] - 0x64

연료레벨 표시 제어

```
void lcd_print(const char *str)
{
    while (*str) lcd_data((uint8)*str++);
}

void ControlFuelLevel(uint8 fuellevel)
{
    static uint8 prev      = 0xFF;

    uint8 pct = (fuellevel > 100) ? 100 : fuellevel;

    if (pct == prev)
    {
        return;
    }
    prev = pct;

    lcd_cmd((uint8)(0x80 | 0x00));
    lcd_print("Fuel Level ");

    char line[17];
    (void)sprintf(line, sizeof(line), "Fuel: %3u%% ", (unsigned)pct);
    lcd_cmd((uint8)(0x80 | 0x40));
    lcd_print(line);

    mcu_printf("[FUEL] Controlling Fuel Level\r\n");
} « end ControlFuelLevel »
```

[실습] VCP-G CAN 메시지 수신 & 센서 제어 통합

1. 제공된 키트와 센서들과 함께 VCP-G와 연결 (회로 참고)
2. 제공된 코드를 참고하여 VCP-G 를 세팅한다.(필요시, make -> fwdn)
3. D3-G와 VCP-G를 Can Transceiver를 사용해 연결한다.
4. 예상결과
 1. D3-G A72 Shell command 에서 IPC example 실행
 2. 좌우 지시등, 비상등, 헤드라이트 조작시, 키트와 연결된 LED가 발광한다.
 3. 시뮬레이터 차량의 연료상태를 키트와 연결된 LCD에서 확인할 수 있다.

[실습] VCP-G CAN 메시지 수신 & 센서 제어 통합

D3-G보드에서

1. `$ cd ~/.home/topst/Fabless_Education_blink/Step0`
2. `$ sudo python3 IPC_Example.py snd --canID 0x101 --sndDataHex 01`

[실습] VCP-G CAN 메시지 수신부 동작 확인

- D3-G의 A72에서 CAN Message 전송 시, VCP 수신부가 호출되는지 확인

- D3-G A72 송신 부

```
topst@TOPST:~/TOPST_MICOM_IPC_Example$ sudo python3 ./IPC_Example.py snd --canID 0x101 --sndDataHex 01
args.mode: snd, args.channel: 1, args.txOnly: 0, args.canID: 0x0101
channel_bitmask: 0x01 (channels: 0)
tx_only_channel_bitmask: 0x00 (channels: None)
canID: 0x0101
sndData: b'\x01'
sndDataLength: 1
Send to ipc file_path: /dev/tcc_ipc_micom
pucData_len: 1
pucData: 01
Calculated CRC: 11AE
Packet size: 14
total size: 16
Packet data before sending:
FF 55 AA 00 05 00 01 00 05 01 00 01 01 01 11 AE
0: sendb'\xffU\xaa\x00\x05\x00\x01\x00\x05\x01\x00\x01\x01\x01\x11\xae' at 1762490193574
example : python3 IPC_Example.py sndrecv --sndData [data]
topst@TOPST:~/TOPST_MICOM_IPC_Example$
```

- D3-G R5 수신 후 송신 부

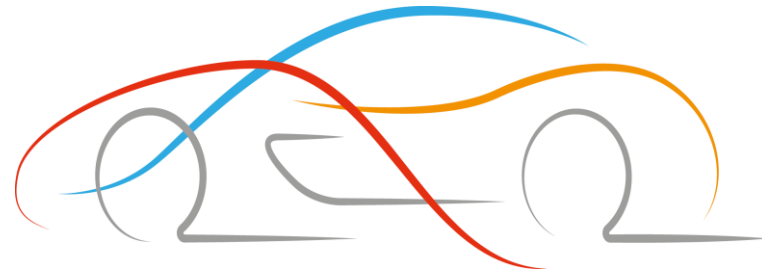
```
(D)[MBOX ][MBOX_DevQget:1314] empty for ch[3] siDevId[2]
(D)[MBOX ][MBOX_DevQget:1314] empty for ch[3] siDevId[2]
(D)[CAN ][CAN_Edu_SetSendingData:1567]
ch_bitmask: 0x1, tx_only_bitmask: 0x0, can_id: 0x101
(D)[CAN ][CAN_Edu_SetSendingData:1583] [CH] : 0x0, [ID] : 0x101, [DATA SIZE] : 1 (5), [DATA] :
(D)[CAN ][CAN_Edu_SetSendingData:1586] 0x01
(D)[CAN ][CAN_Edu_SetSendingData:1591]
```


[실습] VCP-G CAN 메시지 수신부 동작 확인

- D3-G의 A72에서 CAN Message 전송 시, VCP 수신부가 호출되는지 확인
 - VCP-G 수신 부

```
ead@TC-D-20-0546: ~  
MCU BSP Version: V1.0.0  
=====  
CHIP   NAME   : 57045  
DUAL   BANK   : 0  
EXPAND FLASH : 1  
REMAP   MODE   : 0  
HSM    READY  : 1  
=====
```

```
(D)[SAL ][FR_OSStart:504] ~ Done to initialize Free RT Operating System ~  
[ECCP_RecvISR:950]  
[PMIO][PMIO_SetNotiGpkIrq:1118] Set notification of gpk irq for application.  
[PMIO][PMIO_SetNotiGpkIrq:1118] Set notification of gpk irq for application.  
ECCP_DoRx len[6] 0x00 00 00 00 00 00  
[ECCP_CheckHeader:595] Not ECCP frame 00 00 00 00  
[ECCP_DoRx:551] push buff error  
[CAN DEMO]  
< Channel 0 Received Message Information >  
*****  
[ID] : 0x101, [DATA SIZE] : 1, [DATA] :  
0x01  
*****  
[BRAKE] Controlling Brake Light  
[BRAKE] ON  
[CAN DEMO] No message  
|
```



Thank You